11章
データを価値に変える

この章では、多くの人がデータ管理の中で関心を持つ部分について説明します。それは、ビジネスインテリジェンスやアドバンストアナリティクスなどのインテリジェントなサービスによってデータを価値に変えることです。この章では、データを提供する側のアーキテクチャについてはこれまでに説明したことを念頭に置きながら、データを消費する側のアーキテクチャに焦点を当てます。

さっそくですが、筆者の考えを述べさせてください。消費側のユースケースでは、さまざまなドメインやデータプロダクトからデータを組み合わせる必要があります。この作業は技術レベルでも組織レベルでも複雑です。したがって、消費者に焦点を当てたデータは、データプロダクトとは異なる方法で管理することを提案します。ここでの関心事は、データをまとめ、さまざまなビジネスユースケースのために組み合わせることです。データを価値に変え、ユースケースに特化したサービスを利用する、ということです。そこで、この章の大部分をこのトピックに費やします。

データを消費する側は、アーキテクチャの中で最も複雑な部分でもあります。なぜなら、ビジネス上の問題にはそれぞれ独自のコンテキストと要件があるからです。消費側では、多種多様なツール、領域、役割、作業が想定されるため、標準化は困難です。

ビジネス要件は、必ず最優先で考えなければなりません。データをインサイトや行動に変えるには、情報の流れを理解し、その知識を使ってビジネスチャンスを見つけることが必要です。最初は、一度限りのプロジェクトとして始まるかもしれません。しかし、重要なデータに基づいて、組織に対して継続的に価値を提供するソリューションを開発することが理想です。ビジネス要件に応じて、さまざまな技術、ツール、フレームワークを使用することになるかもしれません。

また、非機能要件も考慮する必要があります。多種多様で複雑なユースケースに対応するためには、さまざまなデータベーステクノロジーが必要です。ユースケースに応じて、これらの中から1つまたはいくつかを選ぶことになるでしょう。さらに、データ変換の種類、その速度、並列化のための最適化、消費パターンにも違いがあり、これらはすべて最終的な結果に影響してきます。このほか、ビジネス価値（行動につながるインサイト）を提供するビジネスインテリジェンスと機械学習のパターンにもさまざまなものがあり、それらは複雑で多くのコンポーネントで構成されています。この章では、これらすべてのトピックについて説明します。

これらすべての課題に対する解決策として提案するのは、標準化された再利用可能なパターンやビルディングブロックを作成することです。この方法は、データプロダクトアーキテクチャの管理方法に似ています。しかし、データを消費するためには、さまざまなパターンを定義し、セルフサービスデータと管理対象データを扱うための原則を設定し、データストアの選択、パイプラインの構築、ビジネスインテリジェンスと機械学習サービスの使用に関する注意事項を明確にする必要があります。本章で説明するデータ消費アーキテクチャは、2章で説明したデータ基盤の上に構築され、8章で説明したガバナンスモデルと連携します。

11.1　データを価値に変えることの課題

筆者は、何度も次のような状況を見てきました。具体的な分析機能がなく、スピードが要求される場合、ビジネスユーザーは独自のツールを購入し配備し始めます。ポイントツーポイントのインタフェースを作成し、分析ツールをデータウェアハウスや運用システムに直接接続します。また、自ら管理するビジネス環境に対して、直接データを抽出、変換、読み込みし始めます。新たなユースケースや機会が生まれると、関連する問題を無視して、1つの問題だけを解決するために、これらのツールが次々と導入されていきます。その結果、アーキテクチャは複雑化し、管理が困難になります。

ビジネスユーザーがデータ管理アーキテクチャを混乱に向かわせないためには、基盤となるデータ管理サービスと深く統合した管理環境を構築し、多くのビジネスドメインの連合型モデルをサポートする必要があります。同時に、さまざまな対象者やユーザーグループのニーズにも、**セルフサポート**方式で対応する必要があります。ここでは、この仕組みについて説明します。その前に、

これまで説明したことと、本章で集中的に説明するアーキテクチャについて復習しておきましょう。

これまでは、データをすぐに利用できるようにすることに重点を置いてきました。4章では、データプロダクトの設計と管理について説明しました。その後の章では、APIやイベント管理、メタデータ、マスターデータ管理、ガバナンス、セキュリティなどの領域を掘り下げて、データの流通と管理についてさらに探求しました。これらのトピックはすべて、**図11-1**のアーキテクチャの左側にあるデータプロバイダーの範囲に含まれます。

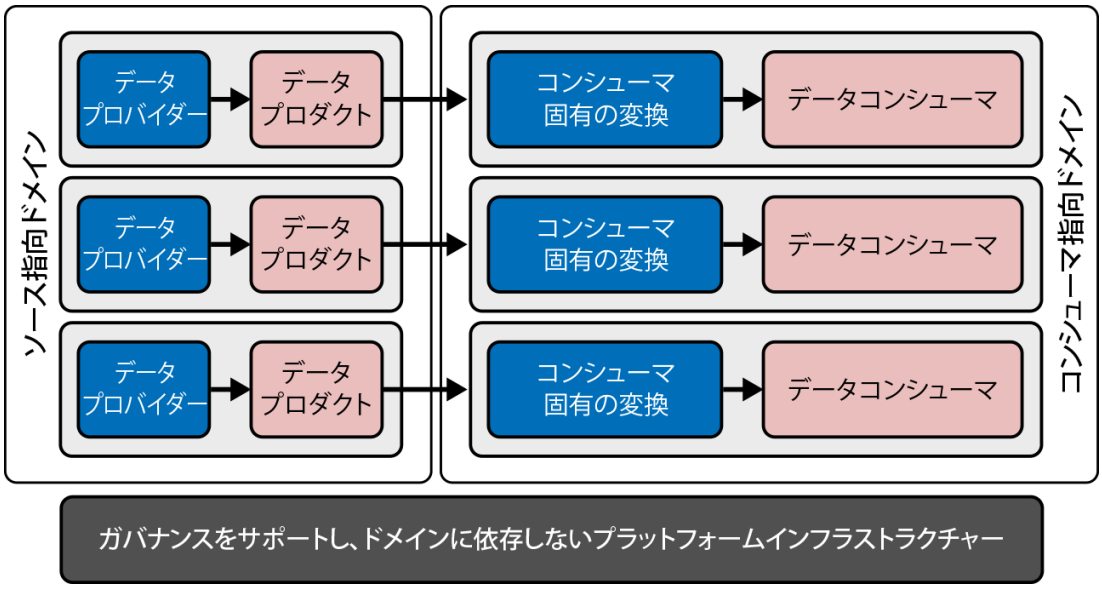

図11-1　データプロバイダーとデータコンシューマの非中央集権的な連携

図11-1のアーキテクチャの右側にあるのは、コンシューマ整合のドメインです。2章で学んだように、これらのドメインは提供ドメインから提供されたデータを消費し、必要に応じて変換してビジネス要件に合わせます。コンシューマ整合のドメインは、一意のコンテキストを持っています。したがって、このコンシューマ固有の変換ステップは**常に**必要となります。データコンシューマは自分たちのユースケースの専門家であるため、このステップはデータコンシューマの責任において行われます。データコンシューマは要件を設定し、どのようなビジネス（分析）目的のために、どのデータを統合しなければならないかを知っています。それでは、データコンシューマは、どのようにこれを行うのでしょうか。

データ消費とそれに付随する変換は、データの複製から始まることが一般的です。しかし、これは必ずしも必要ではありません。実際、大規模なアーキテクチャでは、同じデータのコピーには制限をかけたいはずです。維持にコストと

手間がかかるからです。セキュリティも懸念されます。データが変更されたり、古くなったりする場合もあります。したがって、データコンシューマとして最初に考慮すべきことは、データプロバイダーからのデータプロダクトを、コピーしたり永続化したりすることなく、直接利用できるかどうかということです。

4章で説明したように、データプロダクトは、大量の不変データをデータコンシューマに繰り返し提供するために設計されています。その基本的な性能とアクセスパターンから、クエリ時にデータを直接取得できます（例えば、レポーティング、アドホック分析、アナリティクス、セルフサービス業務など）。直接データを消費するパターンでは、データは読み出されますが、新しいデータが作成されることはありません。消費アプリケーションは、データプロダクトをデータソースとして直接使用します。類似したデータ要素間については、軽量な統合処理を実行する場合があります。アプリケーションはコンテキストを変更し、一時的に新しいデータを作成することはありますが、その結果を別の場所に保存するようなことはありません。この直接消費モデルの大きな利点は、新しい（複雑な）データモデルを作成する必要がないことです。データを抽出し、変換し、新しいデータベースに読み込む必要はありません。変換はその場で行われますが、その結果が新たな保存場所に永続化されることはありません。

このような直接読み出し消費パターンをサポートするには、基盤となるデータプロダクトアーキテクチャが十分な性能を提供する必要があります。しかし、問題は、データプロダクトアーキテクチャでは、コンシューマのニーズを満たすことができない場合があることです。例えば、処理すべきデータ量が、データプロダクトプラットフォームで処理できる量を超えている場合があります。このような場合、（過去）データを新しい場所に少しずつ移して処理を行い、後で利用できるように、あらかじめ最適化しておくという解決策が考えられます。また、複数のデータプロダクトを組み合わせて調和させる必要がある場合にも、問題が発生する可能性があります。通常、多くの構造的な変更、タスクのオーケストレーション、データの連携が必要になるからです。これらのタスクがすべて終了するまでユーザーを待たせることは、ユーザーエクスペリエンスに悪影響を与えます。また、新しいデータを作成する必要がある場合もあります。例えば、複雑なビジネスロジックを使用して新しいビジネスインサイトを生成する場合などです。これらの結果を後の分析のために残しておくには、この情報をどこかに保存しておく必要があります。このような場合は、データ

プロダクトを直接使用するのではなく、ドメインデータストアを作成するという別のデータ消費パターンを使用する必要があります。

11.2　ドメインデータストア

新たに作成されるデータは、ドメインによって、データプロダクトと同じように管理されるべきです。まず、新たなデータを管理するアーキテクチャを、どのような名前で呼ぶべきでしょうか。専門家の中には**データアプリケーション**や**分析アプリケーション**という言葉を使う人もいます。しかし、筆者に言わせれば、これらの言葉はどんな意味にもなり得ます。このアプリケーションではデータプロダクトが消費され統合される、ということを説明しているにすぎません。例えば、分析アプリケーションという言葉は、あらゆる種類のビジネスアプリケーションのことを指すことができます。

2つ目の疑問は、新たに作成されたデータをどのように呼ぶかということです。アーキテクチャの消費側で作成されたデータセットにも、**データプロダクト**という言葉を使う人がいます。しかし、この名前はは誤解を招くと思います。これまで見てきたように、データプロダクトには、ある普遍的な特徴があります。基盤となる提供側のドメインからユビキタス言語を継承していること、作成後も安定性と互換性が保たれていること、アプリケーションから分離されていること、読み出しやすさを重視して設計されていることなどです。これらの特徴は、コンシューマ整合ドメインの一般的な運用方法とは対照的です。特定のユースケースのために変換されたデータは、そのユースケースと緊密に結合しています。必ずしも安定している必要はなく、ユースケースの要件を最適に満たす構造と形式を使う必要があります。その設計は、サポートシステムや使用するデータベース（行指向データベース、時系列データベース、カラムナーストアなど）の仕様に左右されます[†1]。

[†1] **ポリグロット永続化（Polyglot persistence）**とは、データストレージテクノロジーを組み合わせて使用するようなソリューションを表す言葉です。

組織によっては、一意のビジネスニーズを対象としたデータのことを、**コンシューマ整合データプロダクト**と呼んでいます。この用語は、そのデータの範囲と目的を厳密に定義している限り、適切かもしれません。しかし、一般的な用法では、この用語は単に「データプロダクト」と略される可能性が高く、区別がつかなくなり混乱を招きます。

まとめると、ビジネスユースケースのためのデータ消費と統合は、データプロダクトの構築と提供とは異なります。使用されるサービスや技術には重なる部分があるかもしれませんが、根本的な関心事は異なります。データの提供（データプロダクトの作成）は、他者にデータを提供することです。簡単かつ安全に安定した利用を実現するという原則を遵守する必要があります。一方で、データの消費とは、価値創造のためにデータを活用することです。データ駆動型の意思決定を行うためには、多くのツールやサービスを追加しなければならない場合があります。確かに、消費され変換されたデータは、データプロダクトの候補となるかもしれません。しかし、データを価値に変えることが、プロダクトとしてデータを管理することと同じである、という意味ではありません。

このようなデータと、関連するサービスやビジネスユースケースアプリケーションを管理するために、筆者は**ドメインデータストア**（DDS）という新しいビルディングブロックを提案します。4章のデータプロダクトと同様に、データの管理と、対応するアーキテクチャ（テクノロジーサービス）という2つの視点から考えることを提案します。テクノロジーの視点では、データを消費、統合、使用するために必要となる、基本的なソリューションアーキテクチャについて考えます。このレベルでは、データベースなどの製品だけでなく、ETL、BI、MLなどのサービスが必要です。次に、データの視点では、消費されるデータと新たに作成されるデータについて考えます。このレベルでは、データを以下のように分類することができます。

コピーデータ
技術的または構造的な変更が加えられる場合があるものの、元のデータプロダクトのデータと同じ意味を持つデータ†2。

統合データ

消費され、組み合わされ、新しいコンテキストに変換されたデータ[†3]。消費ユースケース用に特別に設計されたものです。

（新たに作成された）データプロダクトのデータ

ほかのドメインで消費できるように統合されたデータ。このデータはデータプロダクトのデータと似ているため、データをプロダクトとして管理するための原則（4章参照）をすべて遵守します。コンシューマ側のデータプロダクトでは、データはすでに説明した構造的な変更を経ており、すべての手順を経た後に共有される点が異なります。

[†2] 前職では、**構文的データ変更**と**意味論的データ変更**という言葉を使いました。構文的なデータ変更では、表現方法は変わるかもしれませんが、データの意味は変わりません。意味論的なデータの変更はコンテキストを変え、その結果意味も変えます。

[†3] 現場のエンジニアの中には、データが処理されていることを明確にするために、**クックドデータ（cooked data）**という言葉を使う人もいます。

メタモデルを使用してデータを分類し、それぞれの種類ごとに原則を設定することができます。例えば、新たに作成されたデータには、必ずリネージが存在しなくてはならないというようなことです。

データの種類によって適用される原則が異なるため、データを正しく分類することが重要です。例えば、統合データはユースケースと密接に結びついているため、ほかのドメインと直接共有すべきではありません。言うまでもなく、統合データに対して破壊的な変更を行うと、データコンシューマに直接影響を与えます。このようなデータをほかのドメインと共有しなければならない場合は、まずデータプロダクトにする必要があります。

消費側全体（テクノロジーとデータの両方）を管理するために、DDSを使用することを提案します。図11-2に示すように、DDSはデータプロダクトストアに似ています。その役割は、コンシューマのユースケース実現のために必要となる、新たに作成されたデータを保管・管理することです。

図11-2のドメインCのように、データをインポートして保存するDDSは、コンシューマがほかのドメインのデータプロダクトを直接使用できない場合にのみ必要です。ドメインの基盤となるテクノロジーは、コスト、アジリティ、知識、スキルのほか、性能、信頼性、データ構造、データアクセス、統合パターンなどの非技術的要件に基づいて、ユースケースごとに選択されます。

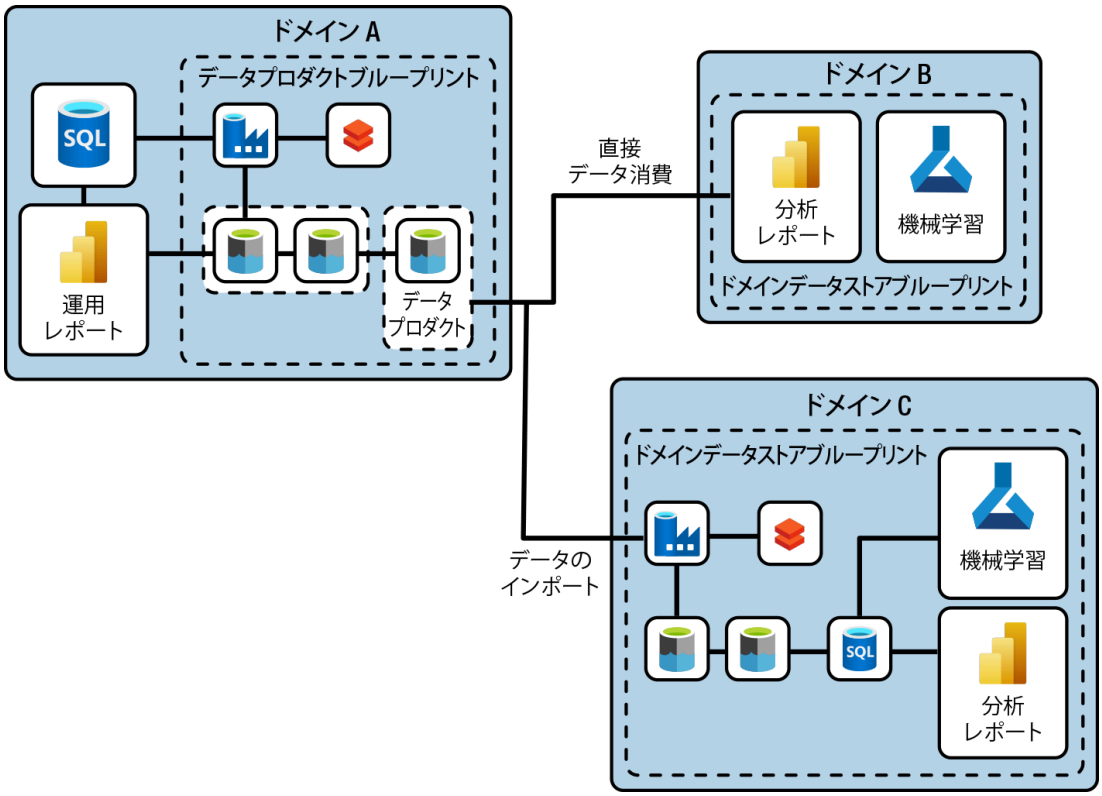

図11-2　ドメインデータストア

DDSはそれ自体がアーキテクチャです。データプロダクトアーキテクチャのように、DDSはブループリントとして提供されます。このため、ドメインチーム用の環境を素早く立ち上げることができます。ブループリントには、データ分析やデータサイエンス用に一連のサービスが配備されている場合もあります。ETLサービス、スケジューリングツール、CI/CDサービスなどが含まれることもあります。チームが必要としないサービスはあらかじめ無効にしてください。

DDSは、データコンシューマとデータプロバイダーの役割を同時に担えます。DDSはコンシューマの役割として消費・統合を行い、そして消費データを新た

に統合データに変換します。さらに、DDSはプロバイダーとして、新たに作成されたデータプロダクトをほかのデータコンシューマに提供します。このプロセスを**図11-3**に示します。

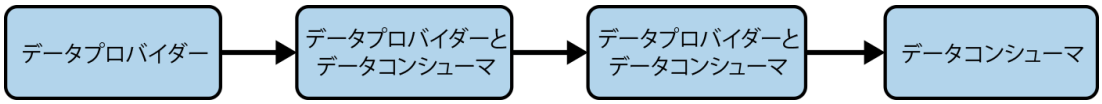

図11-3　データコンシューマがデータプロバイダーになり、データ消費と流通のチェーンが生まれる

アプリケーションを分類するときに、ゴールデンソース（「2.1.3　ゴールデンソース」を参照）とするのか、DDSとするのかは非常に重要です。分類することで、一意で信頼できるデータの説明責任と出所が明確になるからです。また、流通チェーン全体を通じて、データが一意であり不変であることも保証されます。アプリケーションは、その役割に応じて、両方の機能を担うことができます。例えば、あるアプリケーションがほかのドメインからデータを消費する場合、そのアプリケーションをDDSとして分類することができます。一方で、同じアプリケーションが新たにデータを生成する場合、そのアプリケーションは新たに生成されたデータのゴールデンソースの役割を果たすことになります。

データを消費、結合、統合、流通させる場合、DDSの規模と範囲について決断しなければなりません。次の項では、これらの点について説明します。

11.2.1　コンシューマ整合ユースケースの粒度

DDSの数が多すぎたり少なすぎたりしないようにするには、DDSの境界線を明確に設定し、それぞれのユースケースが必要とするデータのみを消費および統合するようにしなければなりません。論理的なDDSの境界の適切な範囲、大きさ、場所を決定することは難しく、ドメイン間でデータを流通させる際に課題が生じます。通常、コンテキスト境界はサブジェクト指向であり、ビジネスケイパビリティとバリューストリームの両方に整合します。ドメインとDDSの境界が整合していることが望ましいのですが、必ずしもそうではありません。非常に大規模なDDSが複数のユースケース（つまりドメイン）で使用されたり、大規模なドメインが同じドメインのさまざまなユースケースのために複数のDDSを持つ場合もあります。

したがって、ドメインの論理的境界を定義する場合、サブドメインを大きなドメインにグループ化します。そして、DDS設計を分解し、より大きなコンテキスト内でのデータモデリング作業や内部データの流通を容易にするほうが良いのかもしれません。例えば、バリューストリーム境界内で複数のドメインが密接に連携している場合、DDSアーキテクチャ境界内で論理的にそれらをまとめます。同じ原則が、データ間で一貫性を必要とするユースケースにも当てはまります。可能であればドメインを分割することも同じように重要です。特にドメインが大きい場合や、サブドメインで汎用的で同じような統合ロジックが必要な場合です。このような状況では、統合ロジックを提供する汎用的なサブドメインを作成し、そのサブドメインをほかのサブドメインが標準として使用することで、そのメリットを享受できるようにするのが効果的です[†4]。サブドメイン間で共有されるモデルは小さいままにしておき、常にユビキタス言語に整合させることが、基本となるルールです。

[†4] ドメイン駆動設計では、ドメインモデルの一部がさまざまなチームやサブドメイン間で共有されることを示すために、**共有カーネル**という用語を使います。共有カーネルを使って統合すれば、重複とオーバーヘッドが削減できます。

分析アーキテクチャを設計する際に重要なことは、DDSの論理的な役割を注意深く考えることです。これには、ビジネスと技術の両方の粒度を考慮しなければなりません。

- ビジネス上の関心事をトップダウンで分解することから始めます。最上位のコンテキスト、スコープ（コンテキスト境界）、活動についての分析です。これらをより小さな機能領域、ユースケース、ビジネス目標に分割する必要があります。この作業には、ビジネスプロセス、ドメイン、機能などをどのように分割するのが効率的かについて、ビジネス知識と専門知識が必要になります。ベストプラクティスは、ビジネスケイパビリティを参照モデルとして使用し、共通の用語（ユビキタス言語）や、共有統合ロジックなどの重複するデータ要件を探すことです。
- 次に、技術的な目標とその達成方法を検討します。再利用性、柔軟性（頻繁に行われる機能変更への順応性）、性能、セキュリティ、スケーラビリティなどです。重要なポイントは、適切なトレードオフを行うことです。2つのビジネスドメインが同じデータを使用していても、技術的な要件が相反

する場合は、関心事を分けた方がよい場合があります。例えば、あるビジネスタスクでデータを大量に集約する必要があり、別のビジネスタスクでは個々のレコードを素早く選択する必要がある場合、たとえ同じデータを使用していたとしても、両者を分離する方が良いでしょう。同じように、あるユースケースでデータ構造を毎日更新する必要があるのに対して、別のユースケースでは少なくとも四半期中は不変にしなければならない場合、両者を分離すべきでしょう。

しかし、話はこれで終わりではありません。例えば、前述したように、データの再利用性に関する懸念についても考える必要があります。同じ統合データが複数のユースケースで必要になるかもしれません。ドメインが大きくなり、複数の（重なり合う）サブドメインで構成されるようになると、ビジネス境界内で内部的にデータをまとめることがさらに複雑になります。このような状況のDDSは、さまざまなドメインゾーンの集合体になる可能性があります。複数のサブドメイン間で共有されるゾーンもあれば、1つのドメインにのみマッピングされるゾーンもあります。メダリオンアーキテクチャを使った例で具体的に説明しましょう。大きなドメインの場合、1つのDDSのさまざまなゾーンの周りに境界線を引くことができます。このDDSの中で、最初の2つのゾーン（ブロンズとシルバー）は複数のサブドメイン間で共有することができます。例えば、クレンジング、修正、履歴データの構築などのタスクはすべてのサブドメインで共通に実行されます。3つ目のゾーン（ゴールド）での変換については、サブドメインやユースケースごとに固有のデータが必要になるため、話が複雑になります。つまり、共有されるパイプラインと、ユースケースに特化したパイプラインが1つ存在することになります。すべてのパイプラインが含まれる、このデータチェーン全体は一体であるため、1つの巨大なDDS実装と見なすことができます[†5]。

[†5] 同様のレイヤリングは、4章で説明したように、データプロダクト開発と（提供）ドメイン境界の確立においても見られたものです。しかし、データプロダクトアーキテクチャとDDSアーキテクチャには明確な違いがあります。データプロダクトアーキテクチャでは、ソースシステムと強く整合しています。DDSでは、統合作業が通常複数のソースにまたがるため、このような整合はありません。

共通の統合モデルを構築した上で、コンシューマ向けに特定の選択や設計を導入するという考え方は新しいものではありません。レイヤリングはデータウェアハウジングのコアとなる概念です。レイヤリングをより大きなドメインに適用する場合、DDSの場合はスコープが明確である点が異なります。DDSでは、境界が明確に設定されています。すべてのデータを統合するのではなく、特定のビジネス目的のためだけにデータを選択し利用します。

消費側を適切にガイドすることは、連合型アーキテクチャをうまく実装するための要件です。ドメインチームにアーキテクチャに関するガイダンスが提供されない場合、複数のドメインで重複した作業が繰り返され、データ結合の際に発生する同じ統合課題に対して異なるソリューションが用いられる危険性があります。したがって、データプロダクトやマスターデータ管理作業と同じように、消費側を管理することをお勧めします。

11.2.2　DDSとデータプロダクトの比較

データプロダクトの目的はデータを提供することです。DDSの目的はその逆です。データを消費し、価値に変えることです。DDSの目的はビジネス上の意思決定を行うことであり、レポーティングや機械学習サービスなどの機能によってサポートされます。DDSで消費されるデータも、データプロダクトアーキテクチャで管理されるデータとは異なります。DDSのデータは、その時々のユースケースに合わせて構造化され最適化されます。ビジネス上の課題はそれぞれ異なり、対象となるユーザーグループ、ビジネス要件、非機能要件も異なります。

しかし、DDSは新たに作成されたデータを流通させるという点では、データプロダクトと重なる部分があります。DDSがデータコンシューマ、データプロバイダー、またはその両方の役割を果たすことがあることはすでに説明しました。つまり、DDSは新たに作成されたインサイトやデータをほかのアプリケーションと共有することができます。これを実現するには、アーキテクチャのデ

ータ提供側とデータ消費側をどのように連携させるかを慎重に検討する必要があります。図11-4に示すように、2つの設計パターンを選択できます。

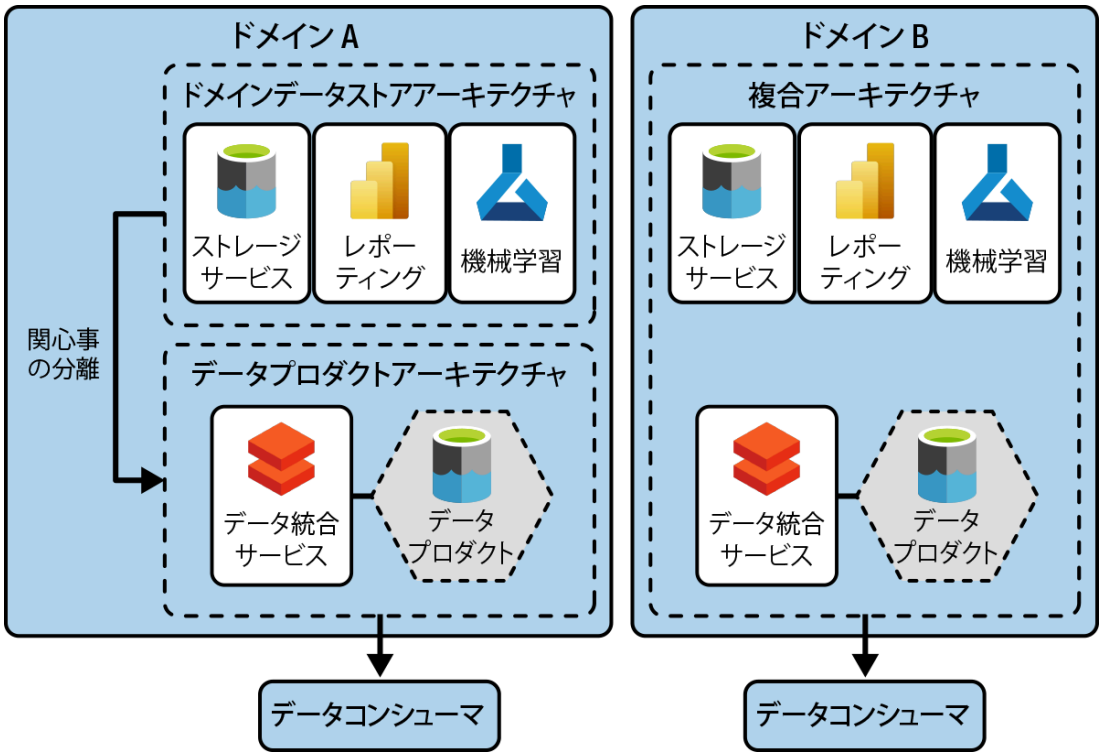

図11-4　DDSとデータプロダクトは、一緒に管理することも、独立して管理することもできる

- ドメインAでは、データの消費と利用で使用するアーキテクチャとは別に、データプロダクト開発用のアーキテクチャを使用します。
- ドメインBでは、データの価値創造とデータプロダクト開発の両方をサポートする複合アーキテクチャを使用しています。

DDSが分析ユースケース用にデータを消費し利用するためだけに設計されている場合、新たに作成されたデータを共有する（新しいデータプロダクトを構築する）には、別のアーキテクチャを使用する必要があります。このシナリオのDDSアーキテクチャでは、新たに一連のサービスを持つ別のブループリントが必要になります。DDSを2つの別々のブループリントで管理するのは、データの利用と共有に関する関心事を分離するためです。この方法では、分析ユースケースの内部アーキテクチャは、データプロダクトを作成・共有するアーキテクチャから分離されます。

DDSがデータプロダクトアーキテクチャを含むように設計されている場合、新たに作成されたデータをほかのコンシューマと直接消費・共有するために同じアーキテクチャを使用することができます。このようにブループリントを設計

すれば、データの消費と共有に使用されるサービスが相乗効果を発揮します。例えば、新たに作成されたデータの消費と共有という2つの目的のために、データ変換用の同じETLフレームワークを使用したり、計算リソースを共有したりすることができます。一方、データコピーと新規作成データが同じアーキテクチャ内で管理されるため、両者を区別することが難しくなります。そのため、例えばメタモデル（9章参照）を使うなどして、リスクを軽減する必要があります。

このように、ブループリントの設計は、DDSとデータプロダクトを一緒に管理するのか、それとも別々に管理するのかによって大きく左右されます。さらに、データを価値に変える作業に何を提供する必要があるかによって、ブループリントが変わってきます。分析ユースケースは一意であり、異なるサービスを必要とする場合があります。例えば、レポーティングサービスのみを利用するユースケースは、機械学習機能を利用するユースケースとではブループリントが異なるでしょう。

DDSの内部で管理されるデータについては、いくつかの基本原則に基づいてデータプロダクトデータと統合データを区別することをお勧めします。統合データとは、これまで学んできたように、消費され、結合され、新しいコンテキストに変換されたデータのことです。入力データは元々別の場所で発生したものであるため、ほかのドメインとのすべての依存関係を示すリネージが大きな関心事となります。データオーナーシップも異なったものになります。データの作成と品質に関する説明責任は統合ドメイン外にあるからです。したがって、統合データを流通させるには、元のデータオーナーの承認が必要です。コピーデータも同様です。元のデータは別の場所にあるため、そのデータのリネージを把握する必要があります。また、統合に関して、重なる関心事を慎重に管理する必要があります。10章で説明した、マスターデータ管理や、集約パターンを使用することを検討してください。

スコープと分離の原則は、BIと分析サービスにも当てはまります。例えば、あるドメインが分析モデルを別のドメインに公開する場合、適切な統合パターンを使って、ドメイン間を分離する必要があります。新たに作成されたデータにも、同じ課題があります。統合データを流通させると、新たな依存関係

が発生します。このため、焦点を絞ってデータを小さなものに分割し、それぞれが特定の1つの問題のみを解決するようにします。共有ドメインモデルは必要最低限に制限します。すべてのチームが依存する分散型モノリスを作らないようにしましょう。また、スコープを明確に設定します。もしデータが集中的に再利用されるのであれば、10章で説明したように、マスターデータ管理の一部となる必要があります。

11.3　ベストプラクティス

企業規模で価値を提供する際の最大の問題は、ユースケースが非常に多様で、組織全体にわたって散在していることです。これらのユースケースは、ビジネス上の問題を解決するためのさまざまな面に対処しようとするもので、対象者も異なります。ユースケースのシナリオは多岐にわたり、それらを実現するために多種多様なデータストアやサービスが必要となります。例えば、あるユースケースではストリーム分析のために頻繁に入力されるデータが必要かもしれませんが、別のユースケースではレポーティングのためにゆっくりと入力される過去データが必要かもしれません。また、プロセスマイニング、画像認識、人工知能を使う場合もあります。また、過去を振り返るだけの説明的・診断的なユースケースもあれば、未来を見通す予見的・戦略的なユースケースもあります。データコンシューマの要求は多様化し続けているため、すべてのユースケースに共通の作業方法や汎用的なソリューション設計を導入することは不可能です。しかし、ビジネス要件、対象ユーザーや運用モデル、データベースやツールなどの非機能要件、データパイプラインやデータモデルに関しては、常に遵守すべきベストプラクティスがあります。この節ではそれぞれについて説明します。

11.3.1　ビジネス要件

出発点は常にビジネス要件です。ユースケースは、ビジネスチームの関心事やビジネスチャンスによって決まります。例えば、インサイトをマネタイズすることで組織に新たな収益をもたらすプロジェクト、顧客満足度を高めるプロジェクト、市場動向を把握することで競争上の優位性を獲得するプロジェクトな

どが考えられます。そのほかのユースケースとしては、例えば、リスク、財務、コスト削減、情況分析、コホート分析と追跡分析、プロセスマイニング、インサイトによる業務効率の改善などが考えられます。また、ビジネス上の意思決定をより正確で予見可能にするためのデータを必要としているユースケースもあります。すべてのビジネスユースケースについて推奨されるのは、長期的なビジネスプランを立てることです。つまり、長期的な付加価値を見据えてください。その場限りのソリューションでその場限りの問題を解決するような状況に陥らないようにしましょう。新しいサービスの必要性を慎重に検討してください。それらは高価で、維持、管理、セキュリティなどが難しいかもしれません。新しいサービスが本当に必要であれば、それを利用できるようにして使用状況を評価した上で、サービスを追加します。どのような場合でも、重複するサービスには注意してください。

ユースケースを実装することは、分析や必要なリソースを考えると、難しいタスクです。準備のために、ビジネスチームとブレインストーミングを行い、すべてのアイデアを整理して分類します。ビジネスモデルキャンバス（https://oreil.ly/49ZSJ）を作成すれば、どのようなビジネスユースケースで、どのようなツールやサービスを活用することで、どのような価値を生み出せるかを明確にすることができます。この情報を、必須ステークホルダー、必要なデータソース、既存のユースケースとの関係、関係者とのやり取り、実装のスケジュールと達成基準などで膨らませていくと良いでしょう。

ビジネス要件によって、必要なデータ量も決まります。例えば、変動が多くばらつきの大きいデータに基づいて機械学習アルゴリズムを開発したい場合、ばらつきの少ない単純なモデルよりもはるかに多くのデータが必要になるでしょう[†6]。

[†6] 機械学習では、より多くのデータはより賢いアルゴリズムに勝るという有名な原則があります（https://oreil.ly/ntW42）。

ビジネス要件を分析した後は、目的と目標が明確に定義され、詳細に記述され、完全であることを確認します。これらを理解することがソリューションの基礎となります。どのようなビジネス上の問題を解決する必要があるのか、どのようなデータソースが必要なのか、どのようなソリューションが運用可能なのか、どのようなデータ処理をリアルタイムやオフラインで実行しなければな

らないのか、完全性と重要性の要件、どんな成果物がほかのドメインで再利用できるのか、などについて明らかにする必要があります。

11.3.2　対象者と運用モデル

分析ユースケースを実現するために、次に行うことは、誰が必要で、どのような責任を負わせるべきかを決定することです。データを価値に変えることに関する役割は数多くあります。これらはさまざまな方法で定義することができ、また誤解されやすいものでもあります。したがって、これらの役割をデータガバナンスフレームワークの一部にすることが、一般的なベストプラクティスです（「8.1.1　ガバナンスフレームワーク」参照）。ここでは、さまざまなペルソナを定義する方法をいくつか提案します。

ドメインオーナー
上位の意思決定者。通常はプロダクトオーナーやビジネス担当者。要件に関してビジネスユーザーと協働し、データ管理ポリシー、プロセス、要件についてガバナンスチームと協力します（8章参照）。

アプリケーションオーナー
アプリケーションや技術ソリューション設計の責任者。すべての開発、セキュリティ、アプリケーション、データの保守活動などを行います。

アナリストや内容領域専門家（SME）
ビジネス要件、データの使用目的、アクセス対象者、データの共有方法を定義する責任者。必要に応じてドメインオーナーや、そのほかのデータオーナーと協力し、ビジネスでのデータ活用をサポートします。データアナリストやSMEは、一般的に「パワーユーザー」と呼ばれます。ツールを使ってデータ分析を行うことはできますが、必ずしもデータサイエンティストやビジネスインテリジェンスの専門家ではありません。

データエンジニア
データの収集、統合、リッチ化を担当。データエンジニアは、データアナリストやデータサイエンティストが適切に業務を遂行できるようにする重要な役割を担っています。データエンジニアはデータパイプラインを管理し、SQLやPythonなどの言語を使用します。

データサイエンティスト
統計学者。通常コンピュータサイエンスや数学の経験があります。データサイエンティストは、アルゴリズム、機械学習、ディープラーニングの使用経験があります。一般的に、ビジネスに精通しており、市場動向も掴んでいます。最終的な目標は、パターンや相関関係を見つけ、データに基づいて予測を立てることです。Python、Scala、R、Javaなどの言語を使用することが多いです。

レポート作成者

ほかのユーザー向けのレポートやダッシュボードの設計、構築の担当者。レポート作成者は通常、優れたビジネスインテリジェンススキルを持っています。

ビジネス上の課題の規模や複雑さに応じて、これらの役割の一部または全部を、機能横断（クロスドメイン）チームで一緒に作業することになります。このようなチームの最適なサイズは、多くの場合、DevOpsチームの規模に相当すると考えられています。5人から10人の間です。小規模で独立したチームを作ることは、人材を共有できないということではありません。ただし、責任の所在は明確に定義すべきです。はっきりと定義され、スコープが設定された目標に集中する小規模なチームを持つことで、多くのチームがビジネスのさまざまな面を同時に推進できるため、規模を拡大することが可能になります。チームメンバー全員が、ビジネス上の課題、対応するソリューション、必要なデータを十分に理解している必要があります。

11.3.3　非機能要件

非機能要件は、ソフトウェアシステムや製品がうまくいくかどうかを左右します。これには、コスト、スケーラビリティと性能、遅延、保守の容易さ、一貫性、セキュリティとガバナンス、書き込みと読み出しの特性、データ量、多様性、速度などが含まれます。それぞれの要件によって選択するものが変わることでしょう。

中心となる問題は、どのようなデータベーステクノロジーやデータストアを使うべきかということです。最適なデータストアを選択することは、多くの条件によって決まります。銀の弾丸は存在しません。また、好みやこれまでの経験も関係します。このため、以下のような問いについて検討してください。

- データの構造はどうなっていますか。データ構造は、前処理工程、クレンジングの種類、モデリングの工程、ストレージの種類などに影響します。この点を最初から理解していないと、無駄な時間を費やし、誤った結果を招くことになります。
- クエリは事前に予測できますか。これは実装に大きな違いを生む可能性があります。クエリが予測できる場合は、キャッシュ、インデックス、事前に準備したデータなどを使って最適化することができます。
- どのような種類のクエリを使いますか。単純な検索、集約、結合、統計処理、テキスト検索、地理検索、そのほか複雑な処理などがあります。例えば、関係データベースは結合をうまく処理できます。

- 現在データ、過去データ、アーカイブデータの要件は何でしょうか。すべて保持する必要があるのでしょうか。
- 完全性と性能のバランスをどうしたいですか。例えば、アプリケーション設計で強一貫性が必要となるのでしょうか、それとも、結果整合性があれば十分なのでしょうか。完全性を重視するのであれば、一般的に、RDBMの方が一貫性を担保しやすくなります。
- 読み出し、インジェスト、完全性のそれぞれの優先度に対して、どのようなトレードオフが存在するのでしょうか。データモデルが異なれば、特性も異なります。例えば、データボルトは設計が柔軟で負荷分散に対応できますが、データの読み出しの負荷は高くなります。
- データベースにどのような操作を行いますか。使用する操作によって、使うものが決まることもあります。分散ファイルシステムは追記には適していますが、（ランダムアクセスの）挿入には適していません。一方、関係データベースはこのような処理を得意としています。
- データのサイズと入出力の速さはどの程度ですか。NoSQLデータベースの中には、バッチ処理のデータインジェストで性能が低下するものがあります。また、高速にデータをインジェスト（メッセージングやストリーミング）する場合、正規化データモデルやACID一貫性モデルは、完全性とロック制御のために性能が低下する場合があります。
- どのようなデータアクセスプロトコルが必要ですか。データベースによっては、SQL、ODBC/JDBC、ネイティブドライバー経由でしかアクセスできないものもあります。また、RESTful API経由でしかアクセスできないものもあります。
- データ構造の変更にどの程度の柔軟性が必要でしょうか。NoSQLデータベースは、スキーマレスにできるため、データ構造の変更にうまく対応できます。
- どの程度のスケーラビリティと弾力性が必要でしょうか。動的な水平スケーリングに対応したシステムもあります。
- データベースの対応言語にはどのようなものがありますか。エンジニアは、これらの言語に精通していますか。習得は難しいのでしょうか。
- ベンダーは特定のストレージソリューションを必要としていますか。ベンダーの要件によって選択肢が制限される場合があります。

そのほか、コスト、オープンソースの標準、特徴、ほかのコンポーネントとの統合、セキュリティ（アクセス制御など）、プライバシー、データガバナンス、開発・保守の容易さなども考慮する必要があります。

組織にどのようなデータストアを選択し、いくつ提供するか、検討してください。ブループリントの一部として、再利用可能なデータベーステクノロジーとパターンの共通セットを定義し、各データストアの強みを最大限に活用できるようにすると良いでしょう。例えば、ミッションクリティカルなアプリケーションや、移行期のアプリケーションには、強一貫性モデルしか使用できないようにします。また、ビジネスインテリジェンスやレポート作成には高速なSQLアクセスを提供可能なデータストアしか使用できないようにしても良いでしょう。包括的な選択肢を提供するよう努めるべきですが、重複は避けてください。何十種類ものデータベーステクノロジーを持つことにならないようにしましょう。

さらに、データベースを選択する際には、ほかのツールやサービスとの整合性も考慮しましょう。例えば、ETLフレームワークの中には、データベースの種類によって、より効果的に機能するものがあります。ブループリントを作成する際にも、これらの点を考慮に入れてください。また、さまざまなクラウド環境ごとに使い分けてみるのもよいでしょう。おそらく、数種類のテクノロジーサービスのリストでほとんどのユースケースが実現できることでしょう。チームが適切に選択できるように、フローチャートや意思決定ツリーを作成することも検討してください。

11.3.4　データパイプラインとデータモデル

データを価値に変える手順としては、データパイプラインを構築し、データを選択し、それをすべて対象となるソリューションに集めることも重要です。ここでの要点は、すべてのデータはすでにデータプロダクトストアに不変的に永続化されているため、余分なコピーを作成する必要は必ずしもないことです。つまり、通常のスケジュールで直接データを取得したり、データプロバイダーから処理が開始できるタイミングを教えてくれるのを待ったりすることになります。データが引き渡された場合、あるコンテキストから別のコンテキストにデータを移動するため、通常、データ変換が必要になります。このコンテキスト変換には、ETLツールが必要です（これについては後ほど説明します）。

リアルタイムのインジェストの場合、メッセージをリアルタイムで取り込んで保存する方法が必要なため、パイプラインの動作が異なります。受信メッセージは、フォルダやデータベースに保存され、後で処理することができます。また、バッファやアプリケーションコンポーネントを使用したり、ストリーミングプラットフォームの統合機能を活用したりすることで、メッセージの分析、

操作、流通を行うこともできます。この話題については、6章を参照してください。

ユースケースの要件に応じて、さまざまなバリエーションが考えられます。例えば、後から分析するためには、データを変換したり、意味づけを適用したりする前に、インジェストされたデータをそのまま保存しておく方がよいでしょう。より高速に処理するためには、ETLバッチ処理とストリーム処理を組み合わせることができます。さまざまなユーザーのニーズに対応するために、さまざまなデータストレージテクノロジーを組み込んだポリグロット設計を使用することもできます。

データパイプラインの構築は、プロジェクト全体の中で特に複雑な部分です。小規模から始めて、柔軟に対応してください。プロジェクトの失敗の多くは、エンジニアが「海を沸かそう（実現不可能なことをしよう）」として、すべてのデータを一度にモデル化しようとしたときに起こります。また、要件をよく考える前に、すぐにテクノロジーに飛びつき、コードを書き始めることもありがちな間違いです。構築し始める前に、すべての機能要件と非機能要件をよく考え、基本的な問いに取り組んでください。再利用可能なタスクはあるのか、すべてのデータを処理するのに最適な順番はどれか、依存関係はあるのか、いくつかのステップを同時に処理できるのか、データをどのようにクエリするのか、などです。

> データサイエンティストは、「80/20 ルール」と呼ばれるものを使うことが多いです。データサイエンティストの時間の80%は、データの検索、クレンジング、整理に費やされ、実際の開発には20%しか残らないというものです。 生データを利用して、最適なモデルを提供できるような単一のアルゴリズムは存在しないからです。

データパイプラインを組み立てる場合は、簡単に再利用したり組み合わせたりできるように、一連の隔離された不変的な変換として組み立ててください。この場合、入力、変換ロジック、出力はそれぞれはっきりと分けておく必要があ

ります。再現性のある出力データを得るためには、すべてのパイプラインをバージョン管理することをお勧めします。最新のインフラストラクチャーの弾力性を活用して性能を向上させるには、パイプラインを並列に実行できるように設計するとよいでしょう。複雑なデータ変換や処理に関しては、Apache Sparkのようなインメモリ分散処理エンジンを使用する方が良い場合もあります。また、開発者にとって使いやすいエコシステムの中で、高水準プログラミング言語で変換ロジックを記述することも検討すべきでしょう。この方法では、良質なコードを記述し共有するためのベストプラクティスが活用できます。

SQLパイプラインとNoSQLパイプラインの比較

一般的に、SQLパイプラインの方が、複雑なクエリをうまく処理し実行できます。ただし、テーブル間の関係をよく理解しておく必要があります。このため、SQLパイプラインでは、多くの検証ステップ、変換・更新ロジックが必要になる場合があります。データを解析し、適切な（制約のある）構造に整える必要があるためです。

一方、NoSQLパイプラインは、もっとシンプルで柔軟性があります。構造を定義することなく、すぐにデータを動的に作成することができます。また、スケーラビリティが高く、多くのテーブルを結合する必要がないため、クエリを高速に実行できます。データは簡単に水平方向に複製できます。ただし、クエリやデータ統合は難しくなります。

データ品質については、データプロダクトの検証機能を利用することができます。しかし、パイプラインに検証機能が必要ないということではありません。データ品質に対するコンシューマの考え方はさまざまなので、パイプラインを拡張して、制約、完全性、正確性、クリーンさなどの、一般的なETLの問題に対処する必要があります。さらに、セキュリティやプライバシーに関する懸念にも対処する必要があります。そのため、データをインポートして処理する前に、さらなるステップが必要になります。

さらに、すべてのETLツールとフレームワークをポートフォリオに分類し、ベストプラクティスと留意点を示すことをお勧めします。柔軟性を持たせながらも、選択肢を限定することで標準化します。例えば、GUIやテンプレートベースのフレームワークは、データの移動や変換がある程度単純なときに使うとよいでしょう。一方、プログラミング指向の言語は、複雑な変換や機械学習が必要な場合に使用するのが良いでしょう。

最後に、パイプライン全体は、リネージ用のメタデータを提供するように構築する必要があります。パイプラインの各ステップで、どのデータがどのように変更されたかを追跡し、理解できるようにします。これには、例えば、データプロダクトへの透過的なポインタが必要です。ファイル名やイベント名、ビジネスキー、ソースシステム識別子などは、すべての変換ステップの正確性と完全性を保証するために、中央のツールで利用できるようにしなければなりません。統合アーキテクチャをすべてつなぎ合わせるために、データカタログを使用するとよいでしょう。また、チームがデータパイプラインを構築する際には、厳格な原則を策定すると良いでしょう。例えば、どのような条件の場合にリネージを中央に流通させる必要があるのかを決めるなどです。このようなリネージに関する要件は、提供するETLツールにも影響を与えるかもしれません。なぜなら、すべてのETLツールが、すべてのリネージソリューションと完全に統合できるわけではないからです。

11.3.5　DDSが果たす役割の検討

さまざまな理由で、下流で消費されるデータを永続化することがあります。まず、いくつか例を挙げ、次にDDSのベストプラクティスを紹介します。

分析データストア
機械学習などの分析ワークロードのデータを統合して結合するためには、分析データストア（ファイルストア）を使う必要があります。目的は、正確なモデル開発のために高品質のデータを提供することです。分析データストアのデータは一般的にフラットにされ非正規化されます。

レポートストア
レポートストアは、ビジネスインテリジェンスや次元レポートに使用されます。一般的に、データはファクト（測定値）と別のコンテキスト用の次元にモデル化されます。最も一般的で簡単な設計としては、スター（https://oreil.ly/NQQGM）や3NFスキーマ（https://oreil.ly/3jVoZ）が選択肢として挙げられます。

集約ストア
10章で説明したように、ドメインのデータ要件が重なる場合、ビジネスロジックの一部をクラスタ化し、新たな集約データプロダクトを作成してドメインで使用するようにした方が良い場合があります。集約データストアは、ドメインを複数のサブドメインに分解でき、データ要件が重複している場合に適した設計方法です。

サードパーティやSaaSソリューション
ユースケースの中には、アーキテクチャの論理的境界の外側に存在する場合もあります。このような状況では、外部コンシューマにデータを提供す

る前に、データを抽出して準備をするためにコネクタとストアが必要になります。

運用コンシューマ

運用コンシューマはOLTPシステムのようなものです。ただし、微妙な差異があります。データを作成するだけでなく、ほかのドメインからデータを消費し統合します。

データウェアハウスやデータレイク

コンシューマがデータウェアハウスやデータレイクサービスを必要とするのは、ほかのドメインからの大量の（過去）データを、作業しやすい共通のフォーマットに組み合わせ統合する必要がある場合でしょう。

すべてのユースケースについて、さまざまなドメインチームに提供するツールやサービスを標準化します。さまざまなシナリオをサポートするために、ブループリントを揃えることを考えましょう。例えば、新たに作成されたデータを流通させるだけの集約ストアは、ビジネスインテリジェンスや機械学習サービスを必要としません。このような場合は、一部の軽量な統合サービスのみを含むような、よりシンプルなブループリントを使用するのがよいでしょう。データストアを絞り込み、チームが選択できるブループリントを整理することで、全体的な複雑さを減らすことができます。連合型では、出発点として必ず汎用のブループリントを使用するようにします。ドメインがデータを消費したりユースケースを構築したりするときには、汎用ブループリントを選択するようにして、ドメインの要件に合わせて（できるだけ最小限の）調整を行い配備します。

ブループリントについては、データプロダクトアーキテクチャとの整合性も考慮してください。例えば、最新のデータスタックから選択するサービスや、データレイクサービスを使用する際に適用するレイヤリングについて標準化すると良いでしょう。さらに、コード配備の自動化、オーケストレーション、ユーザーが配備できるフレームワーク、ライブラリ、ツールなど、ソフトウェア開発の面でも標準化しましょう。最後に、粒度[†7]、命名規則、実体化、権限、データ正規化技術など、データのモデリングと設計に関する標準化を行います。これらすべてを整理するためには、インナーソーシングを利用するか、専門家センターを設立するのが一般的な方法です[†8]。

[†7] 関係の**粒度**は、関係内のそれぞれの行が何を表しているかを定義するものです。`products`（製品）のようなテーブルでは、粒度は1つの製品になり

ます。つまり、1つの製品につき1つの行が存在することになります。粒度をはっきりとさせることで、テーブルのレコードが何を表しているのかを明確にできます。このため、ユーザーはどのようなデータを組み合わせることができるのかを理解できます。

[†8] Nick Tune氏は、インナーソーシングについてのブログ記事（https://oreil.ly/TY1xA）の中で、チーム間の依存関係を適切に管理し排除する方法を説明しています。

標準化という言葉を何度も使っていることにお気づきでしょうか。連合型アーキテクチャへの移行は、標準を設定することから始まります。

データストアによって、データを内部的に管理・整理する方法が異なります。一般的には、インジェスト、クレンジング、キュレーション、調和、提供などの関心事を（論理的または物理的に）分離します。最新のデータストアやデータレイクサービスでは、フォルダ、バケット、データベースなど、さまざまなストレージ技術を使用するゾーンという機能が備わっています。ゾーンは関心事や目的を分離することもできるので、あるストアは消費ユースケースに使用し、別のストアはデータプロダクトの保守や流通に使用することができます。すべてのストアとゾーンについて、そのスコープをはっきりさせる必要があります。複数のコンテキスト境界にまたがってはなりません。各ドメインは自身の境界の中で運用され、その中でデータを消費、統合、新規作成します。また、ゾーンをブループリント設計に組み込むことも検討してください。

インモンとキンボールとデータボルトの比較

データウェアハウスやデータレイクを設計する際、消費側ではどのようなデータモデリング手法を使うべきでしょうか。それはソリューションの特性や要件に大きく依存します。筆者の場合、クラウドベースのデータウェアハウスでは、多くの列を持つワイドでネスト化された非正規化テーブルを使用します。

このようなテーブルを使用すると、クエリの実行が遅くなる原因となる、高価な計算結合や計算ノード間のデータシャッフル[†9]を避けることができます。一方、複雑な統合に関する課題を抱えている場合は、データボルトのような設計が適しています。例えば、多くのソースが同じビジネス概念を使用し関連性があり、一緒に統合する必要がある場合、ハブ・リンク・サテライトを使用することで、関心事をより簡単に分離することができます。さらに、データボルトには、読み込み処理中のテーブル間の依存関係を減らし、挿入のみを実行してインジェスト処理をシンプルにするという利点があります。更新やマージよりも高速に読み出されます。結論としては、各ドメインは、そのユースケース要件に最適なデータモデリング手法を選択すべきです[†10]。

データ駆動型の意思決定ソリューションは、それぞれのユースケースに特化して設計されるはずです。しかし、BIツールやMLサービスは共通していることにお気づきでしょうか。次の節では、それぞれについて詳しく見ていきましょう。

[†9] **シャッフル**は、クエリ実行中に分散テーブルの一部が別のノードに移動するときに発生します。詳細は、Azureドキュメント（https://oreil.ly/QtiCa）を参照してください。

[†10] Christian Kaul氏は、データモデリングに関するリソースについて一覧を作成しています（https://oreil.ly/HYde3）。

11.4　ビジネスインテリジェンス

ビジネスインテリジェンスツールは、ビジネスを支援し、データを構造的にとらえ、深い解釈をもたらします。データへの対話的なアクセスと分析により、意思決定を行えるようになります。レポーティングサービスやダッシュボードサービスなどのBIツールでは、ビジネスに適切な価値を提供するために、考慮しなければならない点がいくつかあります。

11.4.1　セマンテックレイヤー

基本的な問題は、DDSをセマンティックレイヤーで抽象化して、利用可能なデータをユーザーに分かりやすく提示し、簡単かつ一貫した形で使用できるようにするかどうかということです。**セマンティックレイヤー**とは、エンドユーザーが一般的なビジネス用語を使用して自律的にデータにアクセスできるようにする、データのビジネス表現です。セマンティックレイヤーは通常、レポーティングサービスの一部であり、BI開発者やビジネスユーザーによって構築・保守されます。セマンティックレイヤーをビジネスインテリジェンスやそのほかのデータ消費に使用することは多くの利点があります。

- セマンティックレイヤーは、データユーザーがデータをクエリし、データを簡単かつ適切に組み合わせられるようにしてくれます。一般的に、データユーザーは結合について悩む必要はありません。関係はあらかじめ定義されており、誰もが正しい組み合わせを行えるよう支援してくれます。間違った計算や解釈を避けることができます。フィールド名は一貫しており、フォーマットも設定されています。例えば、金額を自動的に合計したり、日付フィールドを時間として計算したりできます。
- セマンティックレイヤーのデータは、全体的な性能を向上させるために、消費に最適化（事前統合、集約、キャッシュ）されています。その結果として、ユーザーエクスペリエンスが大幅に改善され、素早く利用できるようになります。
- 基盤となるデータストアは分離されています。このため、そのデータ構造を変更しても、必ずしもすべてのレポートやダッシュボードにすぐに影響が及ぶわけではありません。
- セマンティックレイヤーには、バージョン管理、行レベルのセキュリティ、監視などの機能を追加することができます。
- セマンティックレイヤーは、**データ仮想化**と呼ばれる技術を使って構築することができます。実装に関する技術的な詳細を、データコンシューマから隠蔽することができます。

セマンティックレイヤーを使って、さまざまな種類のデータ統合シナリオを実現できます。セマンティックレイヤーは、特定の分析ユースケースに整合済みのデータモデルとして、ドメインが直接使用できます。また、ドメインは、ほかのセマンティックモデルの入力としてセマンティックレイヤーを使用することもできます。つまり、あるドメインがほかのドメインのセマンティックレイヤーを自分のセマンティックレイヤーの入力として使用します。この場合、セマンティックモデルは、少し調整を必要としたり、別のドメインのデータと組み合わされたりするのが一般的です。結論を言うと、セマンティックレイヤー

はキュレーションデータを生成するために使われることがあります。つまり、データの意味論的コンテキストが変わる場合がある、ということです。これはデータオーナーシップも変わることを意味します。

ほとんどのBIツールでは、2つの異なる方法でセマンティックレイヤーを構築することができます。最初の方法は、独自のインメモリモデルやキャッシュモデルを使用する方法です。このモデルでは、基盤となるデータストアからのデータは、スケジュールに従ってキャッシュレイヤーに転送（更新）されます。これは、データが複製されることを意味しています。基盤となるデータストアがキャッシュレイヤーよりも新しいデータを持っている場合があるため、データは少し古いものになる場合もあります。

2つ目の方法は、基盤となるデータストアを利用する方法です。このモデルでは、クエリは基盤となるデータソースに渡されるため、セマンティックレイヤーは、クエリに対して直接連合処理を行うメタデータレイヤーに過ぎません。このモデルは、データベースが数秒以内に更新されるため、遅延が少なく、リアルタイムで運用されるレポート作成に適しています。欠点は、レポート構造（通常、次元データモデル）用にデータをモデル化し最適化する必要があることです。

セマンティックレイヤーには欠点があり、注意して使用する必要があります。単純なレポートや予測可能なクエリの場合、オーバーヘッドを考えればレイヤーを追加する必要はありません。また、高度なツールは多くのデータを簡単に収集・保持できるため、最終的にセマンティックレイヤーのコストが高くなるという問題もあります。もう1つの複雑な問題は、レポートと基盤となる（共有）データモデル間の重なりをうまく管理することです。このデータモデルでは、さまざまなソースから同じデータを組み合わせる可能性があります。ランドスケープが大きくなるにつれ、レポートの複雑さも増し、共有データセットへの関係も増えていきます。境界を明確に設定し、自社のケイパビリティと整合させましょう。共有（または複合）モデルは、安定していて本当に重要な場合にのみ使用します。

セマンティックレイヤーとレポートソリューションを構築する際の課題は、データからどのようなインサイトを抽出したいのかをはっきりさせる必要があることです。ビジネス上の問い、組織の目標、データソースを事前に知っておく必要があります。多くの場合、まずデータを評価し、探索し、どのような実行可能なパターンが抽出されるかを知りたいと思うことでしょう。このような理

由から、セルフサービス型のデータ探索ツールや、データプレパレーションツールが人気を集めているのです。

11.4.2　セルフサービスツールとデータ

データ検索、データプレパレーション、データ可視化ツールなどのセルフサービス機能は、データから価値を引き出すプロセスを加速させます。データの探索、結合、クレンジング、変換、可視化のための使いやすく直感的なセルフサービス機能を提供します。データプレパレーションの目的は、レポーティングや分析をユーザーにとってより使いやすく簡単に使用できるようにすることです。つまり、高度な処理と分析に適しています。データアナリストやサイエンティストは、プログラミングや複雑なITスキルを必要とせずに、これらの活動を自分自身で行うことができます[†11]。BIツールやデータ分析ツールの中には、機械学習アルゴリズムが搭載され、データプレパレーションを推奨したり、自動化して高速化したりするものもあります。

[†11] Sean Kandelらは、Wranglerと呼ばれるデータプレパレーションツールについて興味深い学術論文を発表しています（https://oreil.ly/G2X7z）。

データ検索（data discovery）と**データ探索**（data exploration）という言葉を同じ意味で使っている人もいますが、両者は同じではありません。**データ検索**とは、データカタログからデータを検索し、データのパターンやトレンドを発見するために、作業を繰り返すという、より広い意味の言葉です。通常、データ分析に取り掛かる際の最初のステップです。**データ探索**は、データの中に何があるのか、どのような特徴があるのかを、さらに深く理解することです。

データの専門家が自らデータを価値に変えるには、アーキテクチャ設計とブループリントを変更しなければなりません。データ管理ツールとエンドユーザー向けツールをうまく統合しアクセス可能にする必要があるからです。データの専門家は、そのスキルと、必要とするツールによって分類されます。この分類とセルフサービスは、標準化に影響を及ぼします。もし何十ものツールが利用できてしまうと、統合して自動化を適用することは難しくなるでしょう。したがって、セルフサービスの導入を成功させるには、機能が重複するビジネスインテリジェンスツールや分析ツールを削減し、企業特有のニーズに対応する標準化ソリューションの構築を目指さなければなりません。また、管理データとセルフサービスデータを明確に区別する必要もあります。

- 管理データは、現在進行中のビジネスニーズに対応するものであり、その性質上、安定し、自動化され、標準化されたものです。データガバナンスの基準をすべて満たす必要があります。
- セルフサービスデータは、アドホックで1回限りの分析に使用されるため、本来一時的なものです。

セルフサービス処理は、データに何が含まれているのか、どのような関係が考えられるのか、データをさまざまなビジネス用途にどのように利用できるのかについて理解することから始まります。手作業で分析、結合、調整することで、一般的な仮説を検証することができます。これらの活動では、データを使って実験することもあります。検索や探索の成果が明らかになったら、今度は価値を運用する段階です。つまり、管理（本番）環境で、実際のデータを使って、適切に管理されたデータパイプライン（モデルやETL）を構築します。セルフサービスは、ビジネス上のある特定の質問にすぐに答えを出したい、と考えているビジネスユーザーをサポートするためにも利用できます。これは、アドホック分析と呼ばれるものです。

セルフサービスによる作業と、本番ワークロードの間では、必ずビジネスユーザーからデータエンジニアへの引き継ぎが必要になります。セルフサービスは、多くの場合、セマンティックレイヤーの作成や、ビジネスユーザーだけで進められるプロジェクトのために使われます。これは、IT部門が仲介役にならないということです。つまり、ビジネスユーザー自身が分析を推進できるようにします。このため、IT部門は、セルフサービス機能をプラットフォームとして提供することに注力すべきです。この環境におけるデータは一時的なものである可能性があり、厳格な条件下でのみ外部への持ち出しが許可されるべきです。セルフサービスもポリシーで管理できます。軽量で機能を絞った検索環境

を提供している組織も多く見かけます。このような組織では、エンドユーザーは、データパイプラインのオーケストレーションやSparkを使用できないようになっています。

ユーザーがデータをコピーしたり、結果をローカルに保存したりするのを防ぐには、セルフサービス機能を（ブロンズ、シルバー、ゴールドに加えて）、（一時的な）ワークスペースレイヤーにプロビジョニングし、セルフサービスを検索できるようにします。一部の組織では、ワークスペースレイヤーのデータをほかのドメインで検索できるようにしています。しかし、ワークスペースレイヤーにあることで、そのデータは認証されていないとマークされます。このようにして、組織のパワーユーザは、検証、正式な承認、ドキュメント化などを行うことなく、素早くインサイトを発見し共有することができます。

管理データは、セルフサービスデータとは異なり、セルフサービス環境に直接オフロードすることができます。その逆（セルフサービスデータを管理環境に取り込むこと）はできません。管理データは人手に依存してはならないためです。セルフサービスとIT管理環境間の移行は、円滑に行う必要があります。ITを管理する立場にあるユーザーは、セルフサービス探索環境を利用するユーザーと緊密に連携する必要があります。例えば、エンジニアはセルフサービスでの取り組みを検証し、ETLスクリプトのサンプルを作成することもできます。このサンプルは、まずセルフサービス環境でテストされ、最終的には管理環境に移行します。

ビジネスインテリジェンスには、管理データとセルフサービスデータが取り込まれ、意思決定のベースとなります。ただし、組織のニーズに合った適切なバランスを見つけることが重要です。万能なソリューションは存在しないため、一般的なベストプラクティスに従うようにしてください。

11.4.3　ベストプラクティス

ビジネスインテリジェンスに対する投資から最大限の価値を引き出すためには、以下のような取り組みを検討してください。

- BI戦略について目的意識を持った決断を下します。セルフサービスBI、管理型セルフサービスBI、エンタープライズBIの理想的なバランスを決めます[†12]。これらのモデルのオーナーシップと管理を自社のドメインに整合させます。
- ユーザーをサポートするために、プレイブックやデザインパターンを開発します。例えば、BIツールにデータをロードするための最適な方法は何でしょうか。レポートは、データをレポート環境に直接ロードすべきでしょうか。それともまずデータベースを作成し、すべてのレポートをデータベースに接続すべきでしょうか。このような問いに模範解答はなく、設計上の判断を多数行う必要があるため、プレイブックの作成を検討してください。特定の状況に最適と考えられるルールや提案のセットを用意します。
- 専門家センター（COE、center of excellence）の設立を検討しましょう。全社的な標準プロセス、トレーニング、ガイドライン、ベストプラクティス、サポートなどを策定する中央チームのことです。さまざまなビジネスドメインのステークホルダーに話を聞き、データ駆動型の意思決定のために、現在どのプラクティスがうまく機能しており、どのプラクティスがうまく機能していないかを把握します。
- ビジネスインテリジェンスをどのような状況で使用できるかを示すために、成熟度レベルを使用します。例えば、レベル1では、ドキュメント化されておらず、プロセスに関する規律を持たない新しいユースケースに注力します。このレベルでは、正式なプロセスはほとんど存在しません。レベル2は、繰り返し実行可能で、管理されたユースケースに焦点を当てます。ガバナンスモデルが整備され、すべてのユーザーが認証されている必要があります。レベル3は、多くのビジネス価値を提供する重要なユースケースが対象です。このレベルでは、自動化とモニタリングが実施されなければなりません。
- 継続的に知識を構築し、組織内の人々と共有する、認定専門家ネットワークを確立します。
- 提供データのサンプルレポートを提供するようドメインに要求します。これにより、データセットとユースケースへの接続がBIレイヤー内で利用可能になり、コンシューマがそれを発見してアクセスを要求できるようになります。それだけでなく、ダッシュボードを見ることで、ユーザーはデータの意味や結合方法などをすぐに把握できるようになります。

[†12] Microsoftでは、Power BIのドキュメント（https://oreil.ly/GXt4r）の中で、ビジネスインテリジェンスコンテンツを所有および管理する方法について、3種類の戦略を説明しています。

ビジネスインテリジェンスとは、関連するビジネスデータの全体像を明らかにする方法です。同時に、ビジネスインテリジェンスは、多くの場合、過去を振り返り、何かが起こった理由を解明しようとするものです。分析も含まれますが、次に何が起こるかを予測し、自動化された意思決定を行うにはアドバンストアナリティクスが必要です。

11.5　アドバンストアナリティクス（MLOps）

機械学習、**人工知能**、**コグニティブコンピューティング**は最近のバズワードです。これらはすべて重なり合い、補完し合っています。人工知能は、機械が「知的に」動作し、人間が行うようなタスクを機械が実行することの総称です。機械学習は人工知能の一分野であり、広義には、機械が知的な行動を模倣して学習する能力と定義されます。例えば、機械学習モデルは、過去の行動に基づいて、リンクをクリックする確率や何かを購入する確率を予測することができます。コグニティブコンピューティングは、人工知能のまた別の分野です。人間の脳がどのように機能するかに重点を置いており、例えば、音声をテキストに変換したり、画像内のアイテムを認識して分類したりするのに使用できます。

これらのサービスは、訓練や正確なモデルの開発においては使いやすくなってきていますが、本番環境への配備（特に大規模化）は大きな課題となっています[†13]。その理由は以下の通りです.

- モデルはデータパイプラインに強く依存します。データプレパレーション環境から、オフラインデータを使用するのは簡単です。しかし、本番環境では、すべてを自動化しなければなりません。データ品質を保証し、モデルを自動的に再訓練して配備し、新しいデータを使用した後に精度を検証するために人間の承認を得なければならないのです。
- 多くのモデルは通常、スケーラビリティを考慮することなく、隔離されたデータサイエンスのサンドボックス環境で構築されています。さまざまなフレームワーク、言語、インターネットから入手したライブラリ、カスタムコードなどが混ざり合い、組み合わされます。さまざまなチームが存在

し、適切な引き継ぎが行われていない組織構造では、本番環境ですべてを統合することは困難です。

- 本番環境でのモデル管理は、まったく別次元の問題になります。すべてが継続的に監視され、評価され、監査される必要があるためです。リアルタイムで意思決定を行う場合、スコアの有効性、正確性、精度を保証する必要があります。例えば、すべての顧客が無料キャンペーンを受けていたことが2日後に判明するような事態は避けなければなりません。

[†13] Googleの研究者D. Sculley氏らは、機械学習システムにおける隠れた技術的負債について論文（https://oreil.ly/6MaH-）をまとめています。一般的に、実世界の機械学習システムのうち、機械学習コードで構成されている部分はごく一部です。一方で、機械学習システムために必要なインフラストラクチャーは膨大かつ複雑であることが多いのです。

分析サービスの管理という課題を乗り越えるために、コラボレーションとコミュニケーションに関して、新しいプラクティスが生まれました。

機械学習は大規模な管理が困難です。MLのライフサイクルは、データ収集、データ準備、モデルの訓練、モデルの配備、モデルのモニタリング、説明可能性など、多くの複雑なタスクで構成されています。また、ビジネスユーザー、データエンジニア、アプリケーション開発者、機械学習専門家、プラットフォームエンジニアなど、さまざまなユーザーグループ間のコラボレーションも必要です。**MLOps**（「Machine Learning」《機械学習》と「Operations」《運用》の合成語）と呼ばれることの多いフレームワークは、組織が機械学習モデルを本番環境に配備し、それを保守・監視するプロセスを効率化するのに役立ちます。

MLOpsは、データサイエンティストと運用プロフェッショナルのコラボレーションとコミュニケーションに対処し、本番環境のMLライフサイクルの管理を支援します。MLOpsはDevOpsと重なる部分がありますが、ソフトウェアを本番環境に配備することと、分析モデルを本番環境に配備することは根本的に異なるものです。一般的にソフトウェアは静的であるのに対し、MLデータは常に更新されるからです。つまり、分析モデルは常に最新のデータを使って再訓練、再キャリブレーション、再配備されなければなりません。

優れたMLOpsプロセスは、連合型の作業方法と完全に整合しているため、データメッシュアーキテクチャの中にうまく位置づけることができます。エンドツーエンドの作業を管理するために標準的なプロセスを使用することで、大規模な運用が可能になります。このようなフレームワークは、プロセス内の各フェーズが複数の複雑なステージを含む場合にも役立ちます。MLをエンドツーエンドで管理する手助けとして、MLOpsの参照プロセスを示します。このプロセスについては、この後の項で詳しく説明していきます。図11-5は機械学習プロジェクトのライフサイクル全体を示しています。各フェーズにはさまざまなステージがあり、各ステージには複数のステップがあります。

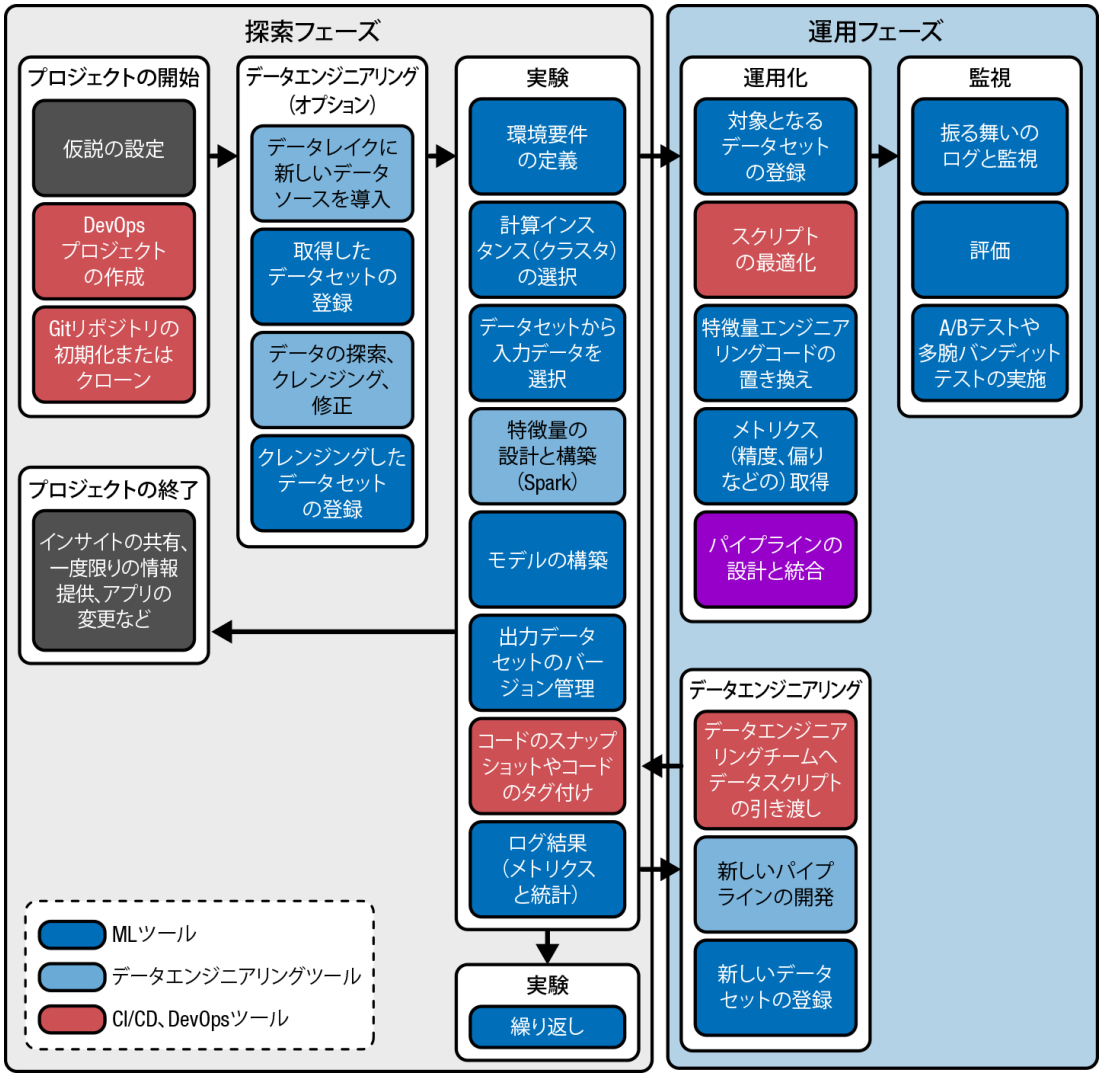

図11-5　機械学習プロジェクトをエンドツーエンドで管理するための推奨MLOpsプロセス

図11-5の左側の探索フェーズから始めましょう。まず、プロジェクトを開始し、ビジネス要件を明確にします。次に、さまざまなアルゴリズムやパラメータをテストしながら、作業を繰り返します。これには、特徴量を作成するために、データエンジニアリング作業が必要になることがよくあります[†14]。プロ

ジェクトの価値を示した後に、新たなフェーズに入ります。本番環境のシステムにモデルを配備します。このフェーズは、**本番運用**や**運用化**と呼ばれることもあります[†15]。これはプロセスの中で最も複雑な部分です。多くの活動が含まれ、データ収集からデータ検証、モデル訓練、モデル検証、モデル配備まで、すべてのステップを管理するために自動化された機械学習パイプラインが必要です。

[†14] **特徴量作成**や**特徴量エンジニアリング**は、データを活用して、訓練データセットにない新しい変数を作成する機械学習方法です。

[†15] データサイエンティストの中には、運用化（operationalization）を「o16n」と略す人もいます。

ユースケースによって、機械学習プロジェクトは異なるものになるでしょう。例えば、同じパイプラインで複数のモデルを訓練、評価、配備する場合もあります。また、1回限りの分析プロジェクトの場合もあります。このような分析プロジェクトでは、分析したインサイトをビジネスと共有しますが、本番環境にはモデルを配備しません。出発点もさまざまです。例えば、データがまだ利用できない場合、まずデータ収集と導入のステップを踏む必要があります。

MLOpsを、より深く理解していただくために、各ステップについて詳しく説明します。その際、ベストプラクティスを共有し、各ステージで得られる成果物についても説明します。最後に、いくつかのバリエーションと例外について説明します。

11.5.1　プロジェクトの開始

MLOpsプロジェクトの最初のフェーズ（**図11-5**の「プロジェクトの開始」ステップ）では、ビジネス要件とデータニーズを理解し、以降のステージで実施する実装と開発作業を計画します。これは、後のステージのための土台となります。まず、ビジネス上の目標を定義し成功基準を策定します。この成功基準が、次のフェーズに進むかどうかを定める条件になります。チームメンバー間のコラボレーションには、DevOpsツールの使用をお勧めします。例えば、カンバンツール（https://oreil.ly/CNibQ）を使えば、プロジェクトのライフサイ

クル全体を通して活動を管理・追跡することができます。これらの活動には、データソースを特定し、データプロダクトが既に開発されているかどうかを確認することが含まれます。

機械学習プロジェクトを始めるときには、すべての（将来の）成果物を保存・管理するために、新しいコードリポジトリを作成することをお勧めします。一般的なベストプラクティスは、ユースケースを互いに分離しておき、新しいMLプロジェクトを作成するたびに、新しいリポジトリを作成するようにします。このような作業を効率的に行うために、コードテンプレートを標準化することを検討してください。コードの再利用を容易にし、プロジェクト開始時や新しいチームメンバーがプロジェクトに参加する際の準備期間を短縮します。

新しいプロジェクトを開始すると、以下のような成果物が得られます。

- ビジネス要件、成功基準、倫理的問題などを網羅したプロジェクトドキュメントやWiki
- データソースや、適用しなければならない要件のリスト
- 初期作業をまとめたチームボード
- テンプレートを使って、プロジェクト開発を加速させる、新しいコードリポジトリ

プロジェクトチームは、これらの成果物を活用して進捗状況を把握することができます。また、将来のプロジェクトのためにインサイトを共有することもできます。ビジネス目標が明確になったら、次のフェーズ「実験と記録」に入ります。

11.5.2　実験と記録

目標とやりたいことを明確にしプロジェクトを立ち上げたら、次は実験です。このプロセスでは、すべての実験結果を記録することをお勧めします。モデルを訓練して実行するたびに、出力だけでなく、すべてのパラメータ、メトリクス、アルゴリズムなどの成果物を記録する必要があります。こうすれば、以下のようなことが実現できます。

- 実験に必要な要素をすべて集め整理します。
- 保存された実験データを使って結果を再現します。
- 時間、データ、フレームワーク、モデル、ユーザーなどに対して、改善を繰り返しログに記録します。

- モデルがどのように開発され、どのようなアルゴリズムが選択され、どのようなデータセットが入力として使用されたかを規制当局に証明します。

実験を記録するためのツールはたくさんあります。人気のあるフレームワークはMLFlow（https://mlflow.org）です。これはフリーなオープンソースであり、AWS、Databricks、Google、Microsoftなど多くの大手ベンダーによって使われています。

実験を行う際には、まずどのような計算インフラストラクチャーと環境が必要かを決めます[†16]。一般的なベストプラクティスは、新しく始めて、クリーンな開発環境を使うことです。実験ごとにすべての作業を記録し、再現性を担保するためにすべての入力と出力をバージョン管理し記録します。すべてのデータエンジニアリング作業に細心の注意を払ってください。これらの一部は一般的なステップであり、ほかのユースケースにも適用できます。最後に、本番環境において、プロジェクトに使用する統合パターンを決める必要があります。これは以下のいずれかになります。

[†16] **環境**とは、機械学習の訓練が行われる環境をカプセル化したもののことです。Pythonパッケージ、環境変数、訓練スクリプトやスコアリングスクリプトに関するソフトウェア設定などのことです。また、Python、Spark、Dockerなどの言語やツールのランタイムも含まれます。

APIモデル
この方法では、モデルはWebサービスとして配備されるため、ほかのアプリケーションやプロセスから利用することができます。API呼び出しを行うことで、モデルから予測が得られます。

バッチイン・バッチアウトモデル
この方法では、モデルはバッチ型（ミニバッチ型）の入力を使って、バッチ型（ミニバッチ型）の予測を出力します。ラベル付きデータのチャンクやサブセットは、訓練、予測、推奨に使われます。入出力の形式は、一般的にCSVやParquetファイルなどのようなファイルのセットになります。

ストリームモデル
この方法では、モデルはデータストリームとやり取りします。データは、少しずつ時間をおいて到着します。追加のデータが必要な場合は、モデルは、DDSのデータベースを直接読み出します。また、モデルは新しいイベントを生成して発行することもできます。

プロジェクトの実験段階では、手作業で進めるのが一般的です。目標は、仮定を検証することにより、仮説を実証することです。この段階が終わると、プロジェクトはその場限りの実習として終了します。ここで得られた学びは共有されます。また、持続可能なビジネス価値が見出されるかもしれません。この場合、次のステップは継続的に価値を提供することになります。そのためには、データ収集、データ検証、モデル訓練、モデル検証、配備のプロセス全体を自動化する必要があります。この点については、データエンジニアリングの活動について述べた後で詳しく説明します。

実験段階が終わるころには、いくつかの新しい成果物ができあがります。以下のようなものが含まれます。

- モデル運用の結果と基準に関するプロジェクトドキュメント。このドキュメントには、モデルレポートも含まれます。モデルレポートとは、試行されたモデルそれぞれについて、実験の詳細を提供する標準テンプレートベースのレポートのことです。
- 分析ワークスペースの新しいオブジェクト[†17]。環境、モデル、データセット、実験や、すべてのログとメトリクスが含まれます。
- 特徴量レポート。新しい特徴量を生成するために使用されるコードへの参照を含んだドキュメント。どのコードが汎用的でほかのユースケースに使用できるのか、どの特徴量が個人の機微データに対して学習されたものなのか、などを分類しておくとよいでしょう[†18]。
- 新しいプロジェクトのために更新されたコードリポジトリやコードテンプレート。
- そのほか改善点のリスト（環境の調整や新しい計算インフラストラクチャーのオプションなど）。

[†17] **ワークスペース**とは、成果物を管理するための最上位リソースです。

[†18] **特徴量**とは、入力データに含まれるデータの列のことです。例えば、ある人の年齢を予測しようとする場合、入力される特徴量は身長や髪の色などです。**ラベル**は、12歳、78歳などの最終的な出力です。

11.5.3　データエンジニアリング

実験やモデル開発中に、データ品質の問題やデータの欠落に直面することがあります。このような問題がほかの場所で起こることがないように、ソースシステムやデータオーナーに報告することが重要です。また、本番環境でモデルを再訓練するために、データパイプラインを開発して、データプロダクトを利用するプロセスを自動化することもお勧めです。そのために、データドリフトの検出（モデルの性能低下）とスキーマ変更に対するコントロールを実現すると良いでしょう。データパイプラインの構築が完了したら、新しく作成したデータセットを公式の成果物として機械学習環境に登録する必要があります。これにより、モデルの開発が加速し、フォルダの場所やユーザー認証情報などの設定に悩まされることがなくなります。

データエンジニアリングフェーズの残りの部分は、特徴量ストアの開発です。**特徴量ストア**は、モデルの訓練や配備のための中央集権的なリポジトリのことです[†19]。定義、ストレージ、特徴量を標準化します。いくつか重要な利点があります。

[†19] 筆者のブログ記事“Feature Stores and Data Mesh”（https://oreil.ly/rNRxe）にも記述したように、筆者としては、中央集中型や共有型のストアはあまり好きではありません。これらはアーキテクチャに不必要な複雑さを加えるからです。

- 開発者は、同じ特徴量や類似の特徴量を再作成するのではなく、特徴量を検索して再利用できるため、特徴量を作成するプロセスが簡素化され効率的になります。
- 複雑さを減らし、性能を向上させることでコストを削減します。例えば、計算コストが高く、複数の機械学習プロジェクトで使用される特徴量があるとします。変換関数を使用して変換した特徴量を複数のトレーニングデータセットに保存するよりも、共有リポジトリに保存する方がはるかに効率的で保守性も高くなります。前処理や特徴量の構築に時間がかかる場合や、特徴量が時間に依存する場合、ほかの特徴量に依存する場合も同様です。

データエンジニアリングフェーズの終了後に、以下のような成果物が新たに追加されます。

- 更新されたソリューション設計。ソース、データパイプライン、統合パターンなどが含まれたアーキテクチャ図や説明です。
- データ収集とクレンジングのための新規または更新されたデータパイプライン。
- 更新されたコードリポジトリには、本番環境で使用可能なすべての成果物が含まれています。
- データオーナーに報告されたデータ品質の問題やデータ欠落の記録。

11.5.4　モデルの運用

本番環境で迅速かつ確実に配備を行うには、機械学習パイプラインを使用して、エンドツーエンドのプロセス全体を自動化する必要があります。この部分は、多くのステップを追加したり、本番アプリケーションと統合するための制約が存在したりするため、非常に難しくなることがよくあります。これらのステップのいくつかを見てみましょう。

- ハードコードされたコマンドライン文、ファイルの場所、ユーザー認証情報などをすべてパラメータや引数に置き換えます。
- 汎用的なデータエンジニアリングのステップをすべて削除し、データエンジニアリングチームに委ねてデータプロダクトのパイプラインに組み込みます。
- データとモデルの両方について、継続的かつ自動的なテスト、訓練、配備を実装します。
- 推論用エンドポイントの単体テストと統合テストを作成して、デバッグを強化し、配備に要する時間を短縮します。
- 特徴量、モデルの精度、可視性、レコード数などのメトリクスを収集するためのステートメントを追加します。
- A/Bテストやシャドーテストを実施し、本番環境に導入する前に偏りと精度を検証します。
- 学んだ教訓やベストプラクティスをほかのチームやチームメンバーと共有します。
- モデルをほかのアプリケーションと統合します。例えば、複合サービスを作成して、さまざまな場所からのデータを組み合わせて統合しなければならないかもしれません。
- 例えば、ONNX（https://oreil.ly/T40p4）のような標準を使用するなど、相互運用性と環境移行のためにフレームワークを使うことを検討してください。フレームワークを使うことで、開発者は多くの環境、スタック、プ

ログラミング言語で出力を処理できるようになります。さらに、開発者グループが、さまざまなMLフレームワークを使用してMLモデルを組み合わせることもできます。
- モデル自体に関する情報を中央モデルカタログに追加します。例えば、訓練時に識別的特徴量が使用されているかどうかや、個人を特定できるデータが使用されているかどうかを記録します。

データサイエンティストが環境構築に多くの時間と労力を費やすよりも、データサイエンティストが効率的に作業できるように、標準的な環境、言語、ライブラリを定義することをお勧めします。実行プラットフォームを常にステートレスに設計することもおすすめですです。一時的なデータを作成したり、参照データを保持してもかまいませんが、長期にわたってデータを永続化しないようにします。実行プラットフォームが使用するデータは、入力フォルダから出力フォルダに書き戻すようにしなければなりません。この点は重要です。データを移動や移行することなく、モデルを簡単に置き換えることができるからです。

この段階が終わると、成果物のリストはさらに長くなります。新たな成果物には、以下のようなものが含まれます。

- モデル、パイプライン、メトリクス、新しいデータセットなどワークスペースオブジェクトの更新
- すべての本番環境の成果物を含む、更新されたコードリポジトリ
- モデルおよびシステムの健全性と主要メトリクスを表示するステータスダッシュボード
- 配備の詳細を記載した最終的なモデリングレポート
- 最終的なソリューションアーキテクチャドキュメント
- データ収集とクレンジングのための新規または更新されたデータパイプライン
- 機械学習アプリケーションと統合するために施した他アプリケーションに対する調整
- コードテンプレートの更新
- プロジェクトの最終ドキュメント

11.5.5　例外

ご覧のように、MLOpsは複雑なプロセスです。さらに問題を複雑にしているのは、さまざまな要因のために汎用アーキテクチャを変更しなければならない

ことです。例えば、以下のような要因が挙げられます。

- パイプラインは、再訓練後に手動での承認を要求したり、データ品質が特定のしきい値に達したときに自動的に停止したりするように設計される場合があります。
- モデルの再訓練は、更新されたモデルの新規リリースプロセス、ビジネスイベント、新しいデータの入力、手作業など、いくつかの方法で開始されます。モデルの配備パイプラインはデータエンジニアリングパイプラインと緊密に結合することもできます。この場合、新しいデータが入ってきたときにモデルが自動的に再訓練され配備されるようにしたいでしょう。
- DDSの設計と構造は、要件に応じて変える場合があります。さまざまなデータ構造や読み出しパターンを有効にするために、ポリグロットデータベース設計を使いたい場合もあるでしょう。
- 偏り、公平性、モデルの品質を監視するために、出力を捕捉し、異常な結果を自動的にチェックする場合があります。このような場合、複数のモデルを同時に実行するようにします
- 説明可能性を高めるためには、すべての学習データとモデルを自動的にバージョン管理しタグ付けすることが必要になるかもしれません。
- モデルの出力は、複数のスコアリング結果をまとめたバッチ経由、ほかのイベントを分析することで生成されるイベント経由、APIエンドポイント（リクエスト/レスポンス形式のWebサービス）経由など、さまざまな方法で提供される場合があります。
- 非構造化データを使う場合、アーキテクチャは異なるものになります。外部サービスやコグニティブサービスを活用したり、外部パーティとデータを共有したりしたい場合もあるでしょう。
- モデルはさまざまな環境に配備され、実行される可能性があります。エンドツーエンドの監視を行うには、機械学習プロセス中に生成されたメタデータを記録、保存、表示、整理、比較、クエリするために、中央メタデータリポジトリを追加する必要があるかもしれません。
- すべてのコグニティブサービスを訓練したり、簡単にカスタマイズできるわけではありません。例えば、OpenAIのChatGPTはそのまま利用することを前提に提供されており、サービスの動作を変更する方法はありません。つまり、偏りを検出したり、長期間にわたってサービスを監視したりすることが難しいかもしれません。どのような種類のコグニティブサービスの使用が許可されるのか、または承認を必要とするかについて、原則を設けることを検討してください。

- モデル自体にAPI呼び出しを行う前に、まずデータを収集・集約するために複雑な複合サービスや読み出しストアを実装する必要があるかもしれません。

ご想像のとおり、これらの要因は、モデルのビジネス要件や、モデルが下流プロセスでどのように消費されるか、下流プロセスにどのように統合されるかに大きく依存します。分析モデルの実装方法には例外や差異が常に存在しますが、汎用アーキテクチャに対して、すべてのベストプラクティスを適用することで、より大規模なスケーラビリティを実現できるでしょう。

11.6　まとめ

データを価値に変えることは困難です。しかし、データ管理と自動化に真摯に取り組むことで、データ利用を大規模に民主化できることはご理解いただけたと思います。この章では、データを消費する側のアーキテクチャに焦点を当てました。さまざまな（直接や、ドメインデータストアを仲介する）消費パターンを見て、DDSを構築するためのベストプラクティスを紹介しました。機能要件と非機能要件の両方を分析することで、サイロをつぶし、DDS間の依存関係を最小限に抑えられることを学びました。また、セルフサービスやデータ管理と、それらを支える原則についても触れました。最後に、多くのデータ駆動型意思決定ソリューションの共通要素である、ビジネスインテリジェンスと機械学習について考察しました。ご覧いただいたように、適切なプロセスを導入すれば、これらの活動をエンドツーエンドで管理することができます。

また、価値実現までの時間を短縮するためには、バージョン管理、分析監視、配備などの作業を手作業で行わないようにすることが重要であることも学びました。これには、従来のデータモデリングをはるかに超える、異なるカルチャーが必要です。データアーキテクトは、十分に訓練された経験豊富なソフトウェアエンジニアになる必要があります。また、自動化を全面的に受け入れることも必要です。以下ようなベストプラクティスが役立つでしょう。

- 重なる関心事を管理する方法をチームに伝えます。境界をどう設定するかに注意してください。境界が細かすぎると、重なったり繰り返しの作業が必要になったりします。逆に、粒度が粗すぎると、結合点が多くなりすぎてしまいます。
- 分析消費パターンをランディングゾーンに整合させます。標準的で再利用可能な機能を実装できるように、ブループリントを作成します。これらの

ブループリントをプレイブックおよびベストプラクティスと整合させます。データ提供機能とデータ消費機能を整合する際に、適切に選択します。

- データモデリングコミュニティを組織内に設置し、チームをガイドし、データが企業標準に沿って変換されるようにします。
- データを提供する側と消費する側に気を配ってください。コンシューマ側では、データの使い方が大きく変化します。これは、より簡単に標準化できる提供側とは対照的です。データコンシューマがデータプロバイダーでもある状況で、この2つの関心事を組み合わせると、分析ソリューションがデータプロダクトとして分類される際に密結合になる危険性があります。したがって、筆者は2種類のブループリントを使用することで、データ利用とデータ流通を厳密に分離することを推奨します。1つはデータプロダクトを作成するためのもので、もう1つはデータを価値に変えるためのものです。
- 構成、ワークスペース、ドメインの整合性を調整します。ワークスペースは、機械学習などの分析サービスが管理されるトップレベルのリソースです。管理性の観点から、ワークスペースと活動内容をどのように整合させるかをマッピングすることが重要です。例えば、チームごとにワークスペースを用意し、チームごとにワークスペースのインスタンスを作成します。このメリットは、すべてを1つの場所で管理できることです。チームメンバーは、結果に簡単にアクセスしたり、探索したり、再利用したりできます。一方で、プロジェクトごとに1つのワークスペースを用意することもできます。この方法の利点は、プロジェクトレベルでコストを管理し、相反する（セキュリティ）関心事を分離できることです。ただし、データ資産が複数のワークスペースインスタンスにまたがるため、データ資産の検索と再利用が難しくなるかもしれません。また、ワークスペースが増えると管理のオーバーヘッドが大きくなることも懸念されます。
- 連携サービスやネットワークの統合方法については、インフラストラクチャーおよびネットワークチームと緊密に協力してください。例えば、ある特定のエンドポイントや連携サービスでは、中央のインフラストラクチャーチームがネットワークやDNSレコードを変更しなければならない場合があります。このような場合、チケットシステムを使用するよりも、自動化を適用することをお勧めします。クラウドベースのイベントをリッスンし、ワークフローをトリガーし、機能を呼び出し、ポリシーを適用するようにします。
- コードリポジトリを使用して、汎用のブループリントテンプレートを実際の実装から分離します。例えば、各チームが独自のリポジトリを持ち、そこでさまざまな環境やカスタマイズなどのパラメータを管理することがで

きます。より高いレベルでは、ポリシー駆動型のガバナンスフレームワークを実装し、リソースの作成を制限したり、設定を変更してリソースをコンプライアンスに準拠させたりすることで、コンプライアンス違反を防止します。

これらの推奨事項に従うことで、アーキテクチャのコスト効率と相互運用性が高まります。また、より一貫した方法でデータを管理することができます。旅も残りわずかとなりました。最後にもう一度ヘリコプターに乗り込み、スピードを上げましょう。この旅の最終章では、これまで学んだことをすべて組み合わせ、理論を実践していきます。最後の空の旅をお楽しみください。